

Embedded Linux for BBB

Complete Study Notes

Beginner-Friendly Reference for BeagleBone Black (BBB)

Based on Udemy Course — Enhanced & Annotated

Topics Covered	Target Board	Level
12 Major Sections	BeagleBone Black	Absolute Beginner

*Topics: Boot Process • U-Boot • eMMC/SD Update • Device Tree • uEnv.txt • initramfs • BusyBox • Buildroot • TFTP/NFS
• Linux Source Tree*

Contents

1. Linux Boot Process Overview	5
1.1 The 4-Stage Boot Chain	5
1.2 Boot Flow Diagram	6
1.3 Stage 1 — RBL (ROM Bootloader)	6
What RBL Does (in order):	6
RBL Summary Table:	8
1.4 Stage 2 — MLO / SPL (Secondary Program Loader)	8
Why Can't RBL Load U-Boot Directly?	9
What MLO Does (in order):	9
1.5 Stage 3 — U-Boot	9
U-Boot's Jobs:	9
Boot Sources Supported by U-Boot:	10
1.6 Stage 4 — Linux Kernel	10
Linux Kernel Boot Sequence (Detailed):	10
2. U-Boot — Complete Guide	11
2.1 U-Boot Source Code Structure	11
2.2 Cross Compilation — Concepts	12
Installing the Cross-Toolchain:	12
2.3 Compiling U-Boot & MLO	12
Method 1: Using the build script (easiest)	12
Method 2: Manual compilation (step by step)	13
2.4 U-Boot Environment Variables	13
Key U-Boot Commands for Environment Variables:	13
Example: Creating and Running a Custom Variable	14
2.5 Manually Booting Linux from U-Boot	14
Step-by-Step Manual Boot from MMC:	14
Decoding the UImage Header:	15
3. uEnv.txt — Automating U-Boot	18
3.1 How uEnv.txt Works	18
3.2 Complete uEnv.txt File Example and Explanation	18
Line-by-Line Explanation:	18
3.3 Creating uEnv.txt via U-Boot Shell (Serial Transfer)	19
4. SD Card Setup for Booting BBB	20
4.1 Required SD Card Partition Layout	20
4.2 Preparing the SD Card Using GParted (Ubuntu)	21

4.3 Copying Files to SD Card.....	22
BOOT Partition Files (copy to /dev/sdb1 mount point):	22
ROOTFS Partition Files (copy to /dev/sdb2 mount point):.....	22
4.4 Booting BBB from SD Card	22
5. Building the Linux Kernel from Source	23
5.1 Download the Kernel Source.....	23
5.2 Kernel Build Steps.....	23
6. Linux Device Tree	25
6.1 What Problem Does the Device Tree Solve?	25
6.2 Device Tree Files and Formats.....	25
6.3 Device Tree File for BBB	25
6.4 Basic Device Tree Syntax	26
6.5 How U-Boot Passes the DTB to the Kernel.....	26
7. initramfs — Initial RAM Filesystem	27
7.1 Is initramfs Required?	27
7.2 What Does initramfs Contain?	27
7.3 Building initramfs — Step by Step.....	27
Booting with initramfs from U-Boot:	28
8. Booting BeagleBone Black over Serial Port.....	30
8.1 How the RBL Selects UART Boot.....	30
How to Force UART Boot:.....	30
8.2 Transferring Files via xmodem	30
Complete Serial Boot Sequence:.....	30
9. TFTP Network Boot (Faster Development Workflow).....	32
9.1 Architecture Overview	32
9.2 Setting Up TFTP Server on Ubuntu.....	32
9.3 TFTP Boot Commands on BBB (at U-Boot prompt).....	33
9.4 Setting Up NFS Root Filesystem	34
NFS Boot Arguments (kernel bootargs):	35
10. BusyBox — Minimal Root Filesystem	36
10.1 Why BusyBox?	36
10.2 Compiling BusyBox	37
10.3 BusyBox Init System	38
Required Directories in rootfs:	39
Loading Kernel Modules at Runtime:	39
11. Buildroot — Automated Embedded Linux Build System.....	40

11.1 What Buildroot Can Build	40
11.2 Getting Started with Buildroot	40
11.3 Configuring and Building for BBB	41
Key menuconfig Settings for BBB:	41
11.4 Deploying Buildroot Output	42
11.5 Enabling SSH in Buildroot	43
12. Linux Source Tree & Detailed Boot Flow	44
12.1 Linux Kernel Source Tree — Key Directories	44
12.2 Detailed Boot Flow: U-Boot to Linux User Space	45
12.3 kernel_init — Finding PID 1	46

SECTION 1 — Linux Boot Process Overview

1. Linux Boot Process Overview

When you power on a BeagleBone Black (or any ARM-based embedded Linux board), the processor does NOT immediately run Linux. Instead, it goes through a chain of small programs called bootloaders, each one handing off to the next, until finally the Linux kernel takes over. This is called the boot chain.

Note

Think of it like starting a car: the ignition key triggers the starter motor, which turns the engine, which then runs on its own. Each step is necessary and happens in a fixed sequence.

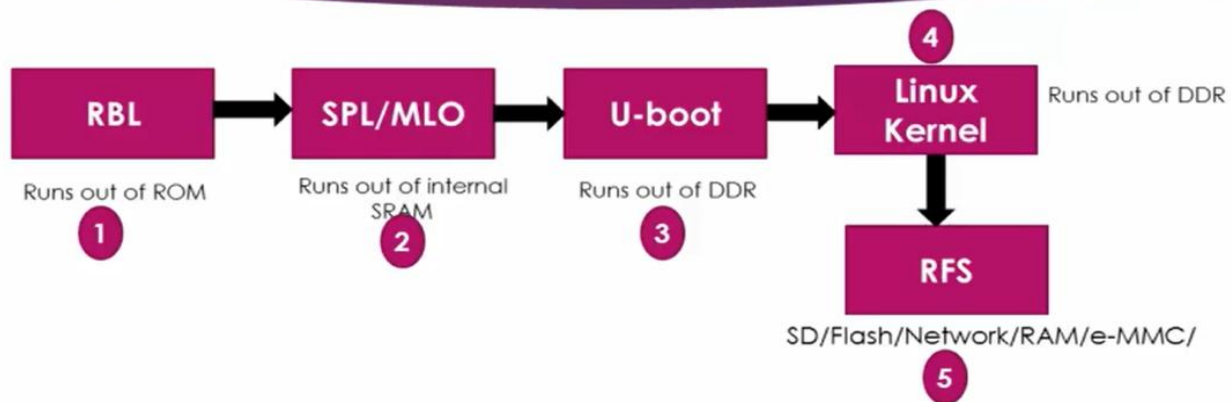


Fig 1.1 — The 4-stage Linux boot chain on BBB. RBL runs from ROM (stage 1), SPL/MLO from internal SRAM (stage 2), U-Boot from DDR RAM (stage 3), then the Linux Kernel loads and mounts the Root File System (RFS) from SD/eMMC/Network (stage 4 & 5).

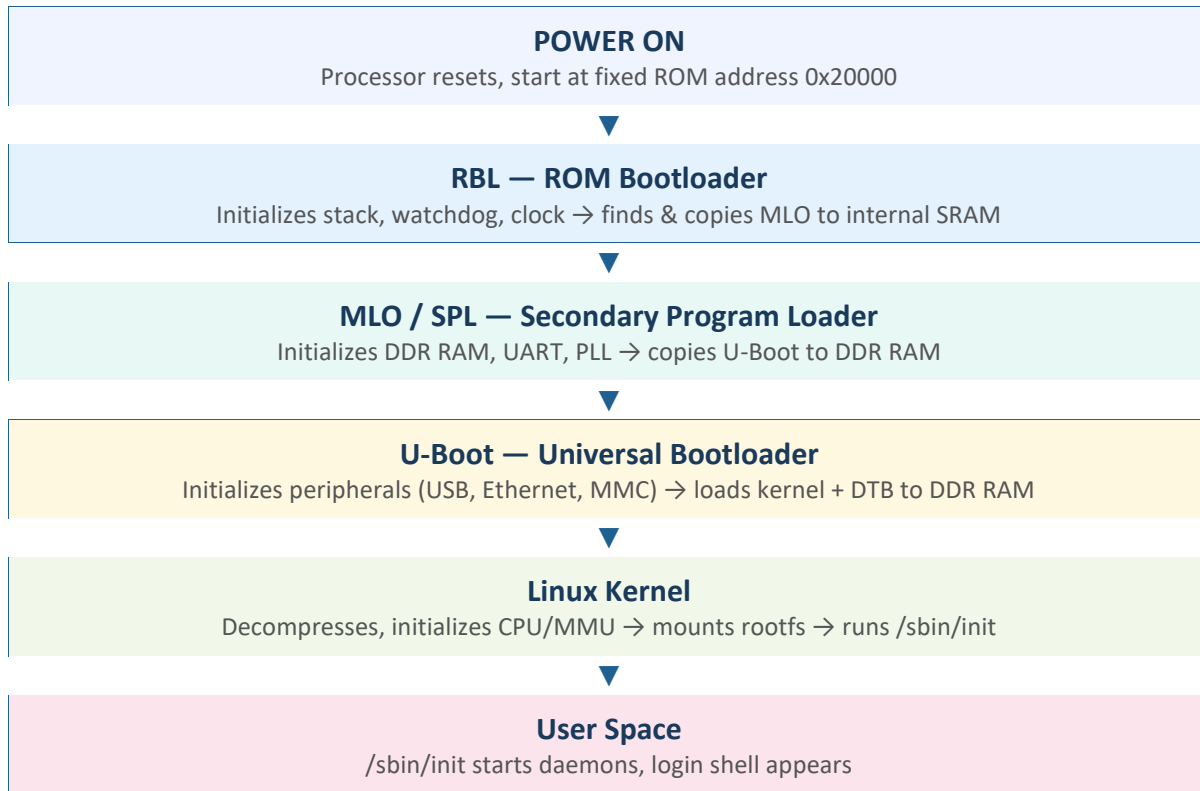
1.1 The 4-Stage Boot Chain

On the BeagleBone Black (AM335x SoC), booting happens in four distinct stages:

Stage	Description
Stage 1: RBL	ROM Bootloader — hardcoded in the SoC's ROM by Texas Instruments. Runs first, always. Cannot be modified.
Stage 2: MLO / SPL	Memory Loader / Secondary Program Loader — a tiny program that initializes RAM and loads U-Boot.
Stage 3: U-Boot	Universal Bootloader — a full-featured bootloader that initializes peripherals and loads the Linux kernel.
Stage 4: Linux Kernel	The actual Linux operating system starts, mounts the root filesystem, and runs user applications.

1.2 Boot Flow Diagram

Each box below represents a stage that runs in sequence on every power-on:



1.3 Stage 1 — RBL (ROM Bootloader)

The RBL (ROM Bootloader) is the very first piece of code that runs when the SoC gets power. It is burned permanently into the chip's internal ROM by Texas Instruments and cannot be changed.

What RBL Does (in order):

1. Sets up the CPU stack in the small internal SRAM.
2. Calls the bootloader main() function.
3. It initializes and starts Watchdog Timer 1 with a 3-minute timeout. If MLO is not loaded within 3 minutes, the SoC resets automatically.
4. Configure the system clock using the PLL (Phase Locked Loop) engine.
5. Reads boot configuration pins SYSBOOT[14:15] to determine clock source frequency.
6. Scans boot devices in order (MMC, USB, UART, SPI...) to find MLO.
7. Copies MLO from the boot device into the internal SRAM of the SoC.
8. Jumps to the start of MLO in SRAM.

26.1.4.3 Clocking Configuration

The device supports the following frequencies based on `SYSBOOT[15:14]`

Table 26-5. Crystal Frequencies Supported

<code>SYSBOOT[15:14]</code>	Crystal Frequency
00b	19.2 MHz
01b	24 MHz
10b	25 MHz
11b	26 MHz

 Fig 1.2 — Crystal Frequencies Supported (from TI AM335x TRM, Table 26-5). This table shows how `SYSBOOT[15:14]` pin configuration determines which crystal frequency the RBL expects. BBB uses `SYSBOOT = 01b` (pin14=VCC, pin15=GND), which selects 24 MHz. The PLL then uses this 24 MHz reference to generate the 500 MHz CPU clock.

<code>SYSBOOT[15:14]</code>	Crystal Frequency
00b	19.2 MHz
01b (BBB default)	24 MHz
10b	25 MHz
11b	26 MHz

Note

PLL (Phase Locked Loop) is a hardware circuit that takes a low-frequency input clock (from a crystal oscillator — typically 24 MHz on BBB) and multiplies it up to a much higher frequency for the CPU and peripherals. The RBL configures the PLL to run at 500 MHz for the BBB's default 24 MHz crystal.

Note

`SYSBOOT[14:15]` are physical pins on the SoC. On BBB: pin 14 → VCC (high), pin 15 → GND (low). This combination signals the PLL to use a 24 MHz reference and output 500 MHz. This frequency cannot be changed during RBL — only MLO can later reconfigure it.

RBL Summary Table:

Task	Details
Stack Setup	Sets up initial CPU stack in internal SRAM for C code execution
Watchdog Timer	Started with 3-minute timeout as safety reset mechanism
Clock Init	Reads SYSBOOT pins → programs PLL → 500 MHz output
Boot Device Search	Scans MMC0 → MMC1 → USB → UART → SPI for MLO/SPL file
Copy MLO to SRAM	Copies MLO binary (max ~256 KB) into internal SRAM
Jump to MLO	Transfers CPU execution to MLO start address in SRAM

⚠ Important

The RBL always looks for a file named exactly 'MLO' on the boot device.

On a FAT-formatted SD card, this file MUST be in the root directory and named 'MLO'.

Any other name will cause the RBL to fail and the watchdog will reset the board.

1.4 Stage 2 — MLO / SPL (Secondary Program Loader)

MLO stands for Memory Loader (or SPL — Secondary Program Loader). It is a small program (~100–250 KB) that bridges the gap between the RBL and U-Boot. The key problem it solves: U-Boot is too large (500 KB to 1.5 MB) to fit in with the SoC's internal SRAM (~128 KB available). MLO initializes the external DDR RAM, so U-Boot has somewhere to live.



SPL and MLO are almost same except the fact that, MLO has a header which contains some info like load address, size of the image to follow, etc

Fig 1.3 — MLO vs SPL structure. Both MLO and SPL contain the same SPL executable code (shown in magenta/purple). The difference is that MLO has an Image Header prepended (shown in yellow), which contains load_addr=0x40000000 (where to

load the image in SRAM) and the Size of the image. RBL reads this header first to know where to place the image in memory, then copies the SPL code there.

Why Can't RBL Load U-Boot Directly?

Resource	Details
Internal SRAM size	~128 KB total (some reserved for stack and secure regions — effectively < 100 KB usable)
U-Boot typical size	500 KB to 1.5 MB — doesn't fit in SRAM
DDR RAM at RBL time	DDR RAM controller not yet initialized — DDR cannot be used yet
MLO solution	MLO fits in SRAM, initializes DDR, then copies U-Boot to DDR

What MLO Does (in order):

9. Initializes UART console — so you can see debug messages from this point onwards.
10. Reconfigures PLL if a different clock speed is required (optional, board-specific).
11. Initializes the DDR RAM controller so DDR memory becomes usable.
12. Configures pin multiplexing (MUX) many SoC pins serve multiple purposes; MLO sets each pin to the function needed for booting (e.g., MMC pins).
13. Reads U-Boot image from the boot device (SD card, eMMC, etc.).
14. Copies U-Boot to DDR RAM at the address specified in U-Boot's header.
15. Jumps to U-Boot's entry point in DDR RAM.

Tip

Pin Multiplexing (MUX): A single physical pin on the SoC can be connected to different internal hardware signals (SPI, CAN, I2C, MMC etc.) but only one at a time. MLO configures the MUX registers to connect each pin to the right peripheral before trying to communicate over that peripheral.

1.5 Stage 3 — U-Boot

U-Boot (Universal Bootloader) is an open-source, feature-rich bootloader used on millions of embedded devices worldwide. It is the third stage and the most configurable one.

U-Boot's Jobs:

- Initializes hardware peripherals: I2C, NAND flash, Ethernet, USB, UART, MMC, etc.
- Provide an interactive command-line interface for debugging and manual booting.
- Load the Linux kernel image (ulmage or zlmage) from a boot source into DDR RAM.
- Load the Device Tree Blob (DTB file) into DDR RAM.
- Set up kernel boot arguments (bootargs environment variable).
- Jump to the kernel entry point.

Boot Sources Supported by U-Boot:

U-Boot can load the kernel from: SD card (MMC0), eMMC (MMC1), USB, Ethernet (TFTP), Serial Port (xmodem), NAND flash.

1.6 Stage 4 — Linux Kernel

After U-Boot loads the kernel into DDR RAM and sets the boot arguments, it calls the kernel's entry point. The kernel takes over from here.

Linux Kernel Boot Sequence (Detailed):

16. U-Boot calls `kernel_entry(0, machid, r2)` — `r2` holds the address of the DTB in DDR RAM.
17. `arch/arm/boot/compressed/head.S` runs — sets up basic environment in Thumb/ARM mode.
18. `misc.c` decompresses the kernel image (`zImage` is a compressed kernel).
19. `arch/arm/kernel/head.S` runs — performs CPU-specific initialization.
20. `head.S` validates the processor architecture, sets up page tables, initializes and enables the MMU (virtual memory).
21. `head.S` calls `start_kernel()` in `init/main.c`.
22. `start_kernel()` initialises all kernel subsystems: memory, scheduler, IRQ, filesystems...
23. `start_kernel()` calls `rest_init()` which creates two kernel threads: `kernel_init` and `kthreadd`.
24. The scheduler starts running. The idle task (`while(1)`) runs when nothing else is scheduled.
25. `kernel_init` frees the memory used for boot-time init functions (marked `__init`).
26. `kernel_init` tries to execute the first userspace program in this order: `/sbin/init` → `/etc/init` → `/bin/init` → `/bin/sh`. The first one that exists and runs successfully becomes PID 1.

Note

MMU = Memory Management Unit. It translates 'virtual addresses' that programs use into 'physical addresses' that actually exist in RAM. Before MMU is enabled, every address is a direct physical address. After MMU is on, processes get isolated address spaces and can't accidentally overwrite each other's memory.

SECTION 2 — U-Boot Deep Dive

2. U-Boot — Complete Guide

U-Boot is the most important bootloader in the embedded Linux world. This section covers its source structure, environment variables, commands, and how to compile it from scratch.

2.1 U-Boot Source Code Structure

Clone URL: <https://openbeagle.org/beagleboard/u-boot/-/tree/v2022.04-bbb.io-am335x-am57xx>

Directory/File	Purpose
arch/	Architecture-specific code (ARM, MIPS, x86, RISC-V, etc.)
arch/arm/cpu/	ARM CPU-specific code
arch/arm/cpu/arm7/start.S	Entry point: the assembly file that jumps from ROM to SPL
arch/arm/lib/bootm.c	Kernel boot code — picks kernel from RAM and hands over control
board/	Board-specific files for hundreds of hardware platforms
board/ti/am335x/	Files specific to the BeagleBone Black (AM335x SoC)
configs/	Default configuration files for each supported board
configs/am335x_boneblack_defconfig	Default config file for BBB — used during build
include/	Header files and board configuration headers
common/	Common code: command implementation, environment handling
drivers/	Device drivers: MMC, USB, Ethernet, serial, etc.
Makefile	Top-level build system entry point

2.2 Cross Compilation — Concepts

Since your development PC runs on x86/x64 but BBB uses ARM, you cannot compile U-Boot with your PC's native GCC. You need a cross-compiler — a compiler that runs on x86 but produces ARM binary code.

Term	Meaning
Host machine	Your PC (x86_64 Linux)
Target machine	BeagleBone Black (ARM Cortex-A8)
Cross compiler	GCC that runs on host but generates code for target
CROSS_COMPILE prefix	arm-linux-gnueabi- (hard-float ABI) or arm-linux-gnueabi-
Toolchain	Set of tools: compiler (gcc), linker (ld), assembler (as), debugger (gdb)

Installing the Cross-Toolchain:

Download from: <https://releases.linaro.org/components/toolchain/binaries/latest-7/arm-linux-gnueabi/>

```
# Extract the toolchain
tar -xf gcc-linaro-7.5.0-2019.12-x86_64_arm-linux-gnueabi.tar.xz

# Add to PATH permanently — edit your shell profile:
vi ~/.bashrc
# Add this line at the end:
export PATH=$PATH:/home/user/gcc-linaro-7.5.0-2019.12-x86_64_arm-linux-gnueabi/bin

# Apply changes:
source ~/.bashrc

# Verify installation:
arm-linux-gnueabi-gcc --version
```

2.3 Compiling U-Boot & MLO

Method 1: Using the build script (easiest)

```
# Give execution permission
sudo chmod +x build-am335x.sh

# Run the script
sudo ./build-am335x.sh
```

Method 2: Manual compilation (step by step)

Step 1: Clean previous build artifacts

```
make CROSS_COMPILE=arm-linux-gnueabihf- distclean
```

This deletes all previously compiled object files so you start fresh.

Step 2: Apply board default configuration

```
make CROSS_COMPILE=arm-linux-gnueabihf- am335x_boneblack_defconfig
```

This reads the defconfig file from configs/ and writes a .config file.

Output: 'configuration written to .config'

Step 3: Optional: Open menuconfig for customisation

```
make CROSS_COMPILE=arm-linux-gnueabihf- menuconfig
```

A text-based GUI opens. You can enable/disable features. Press Escape when done.

Step 4: Compile with parallel jobs

```
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- -j4 # for 4-core CPU
```

```
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- -j8 # for 8-core CPU
```

The -j flag tells 'make' to run N compilation jobs in parallel (speeds up build).

After successful compilation, these files are generated:

- u-boot.img — the U-Boot bootloader binary
- MLO — the SPL/Secondary Program Loader
- spl/u-boot-spl.bin — SPL binary (same as MLO)

2.4 U-Boot Environment Variables

U-Boot maintains a set of environment variables that control its behaviour. These can be viewed, created, and modified at the U-Boot prompt.

Key U-Boot Commands for Environment Variables:

Command	What it Does
printenv	Lists ALL environment variables and their values
printenv bootargs	Shows the value of a specific variable (bootargs in this example)
setenv serverip 192.168.7.1	Creates or updates an environment variable
setenv bootargs console=ttyO0,115200	Sets the kernel boot arguments
run bootcmd	Executes the commands stored inside the bootcmd variable

Command	What it Does
saveenv	Saves all current environment variables to persistent storage
env import -t 0x82000000 36	Imports environment variables from memory address 0x82000000 (36 bytes)

Example: Creating and Running a Custom Variable

```
# Create a variable that holds a command:
setenv my_mmc_list 'mmc list'

# Check it was saved:
printenv my_mmc_list
# Output: my_mmc_list=mmc list

# Run the command stored in the variable:
run my_mmc_list
# Output:
# OMAP SD/MMC: 0 (SD)      <-- SD card slot
# OMAP SD/MMC: 1          <-- eMMC storage
```

2.5 Manually Booting Linux from U-Boot

You can boot Linux manually from the U-Boot prompt without any uEnv.txt file. This is useful for debugging. Press any key when you see 'Hit any key to stop autoboot 3' to enter the U-Boot shell.

Step-by-Step Manual Boot from MMC:

```
# Step 1: List of available MMC devices
mmc list
# OMAP SD/MMC: 0 (SD)  <-- mmc 0 = SD card
# OMAP SD/MMC: 1      <-- mmc 1 = eMMC

# Step 2: Load kernel image into DDR RAM
# Syntax: load mmc <device>:<partition> <address> <filename>
load mmc 0:1 0x82000000 uImage
# '0:1' means device 0 (SD card), partition 1 (BOOT partition, FAT)
# '0x82000000' is the DDR RAM target address

# Step 3: Load Device Tree Blob (DTB) into DDR RAM
load mmc 0:1 0x88000000 am335x-boneblack.dtb

# Step 4: Set kernel boot arguments
setenv bootargs console=ttyO0,115200 root=/dev/mmcblk0p2 rw
# ttyO0 = UART0 serial console at 115200 baud
# root=/dev/mmcblk0p2 = mount partition 2 of SD card as root filesystem

# Step 5: Boot the kernel
# Syntax: bootm <kernel_addr> - <dtb_addr>
bootm 0x82000000 - 0x88000000
# The '-' in the middle is a placeholder for initramfs (not used here)
```

Note

Address Map for BBB DDR RAM:

0x82000000 → Load kernel image (ulmage) here

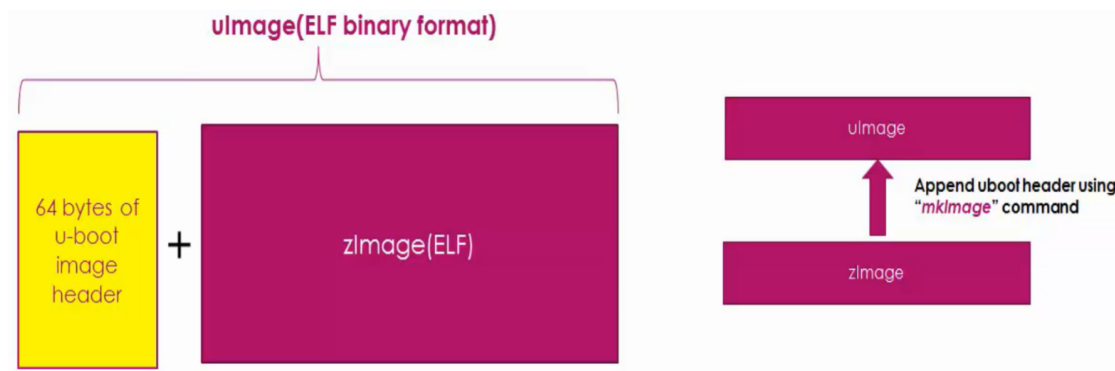
0x88000000 → Load Device Tree Blob (.dtb) here

0x88080000 → Load initramfs here (if used)

These are conventions — you can use other addresses if they don't overlap.

Decoding the UImage Header:

A ulmage is a zImage (compressed kernel) with a 64-byte U-Boot header prepended. You can inspect this header:

ulmage Structure

S

Fig 2.1 — ulmage structure. A ulmage (ELF binary format) is created by taking the standard zImage (the compressed Linux kernel ELF binary) and prepending a 64-byte U-Boot image header using the 'mkImage' command. The header tells U-Boot: where to load the image in DDR RAM, the image size, CRC checksum, architecture type, and entry point address.

Uboot Linux Image header

```
/*
 * Legacy format image header,
 * all data in network byte order (aka natural aka bigendian).
 */
typedef struct image_header {
    uint32_t    ih_magic;        /* Image Header Magic Number */
    uint32_t    ih_hcrc;        /* Image Header CRC Checksum */
    uint32_t    ih_time;        /* Image Creation Timestamp */
    uint32_t    ih_size;        /* Image Data Size */
    uint32_t    ih_load;        /* Data Load Address */
    uint32_t    ih_ep;          /* Entry Point Address */
    uint32_t    ih_dcrc;        /* Image Data CRC Checksum */
    uint8_t     ih_os;          /* Operating System */
    uint8_t     ih_arch;        /* CPU architecture */
    uint8_t     ih_type;        /* Image Type */
    uint8_t     ih_comp;        /* Compression Type */
    uint8_t     ih_name[IH_NMLEN]; /* Image Name */
} image_header_t;
```

Fig 2.2 — U-Boot Linux Image Header C structure (image_header_t). This is the actual C struct definition from U-Boot source code (include/image.h). Each field is a 32-bit or 8-bit integer stored in big-endian (network byte order). ih_magic identifies the file as a U-Boot image, ih_hcrc verifies the header itself, ih_dcrc verifies the data payload, ih_load is where to put the image in RAM, ih_ep is the first instruction to run (entry point), ih_name is a 32-character human-readable name.

Manually Reading the uImage Header at U-Boot Prompt

After loading uImage to DDR RAM at 0x82000000, you can inspect its header two ways:

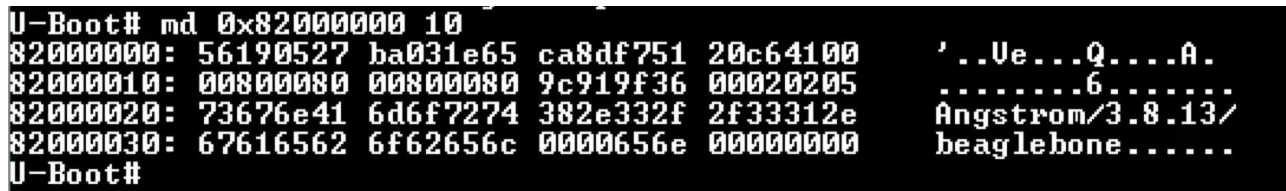


```
md - memory display

Usage:
md [.b, .w, .l] address [# of objects]
U-Boot# md 0x82000000 4
82000000: 56190527 ba031e65 ca8df751 20c64100  '..Ue...Q....'
U-Boot#
```

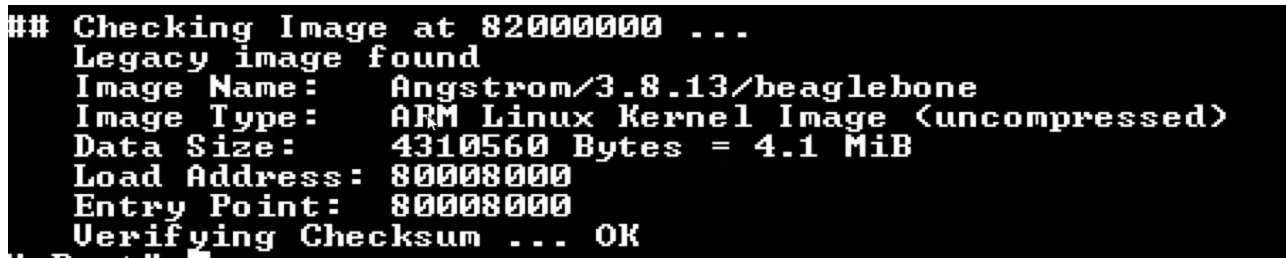
Magic Number
CRC checksum
Image creation Time
Size of Image

 Fig 2.3 — Using 'md' (memory display) command at U-Boot prompt to read raw header bytes. Command: 'md 0x82000000 4' displays 4 words (16 bytes) starting at 0x82000000. The four highlighted values are (in order): Magic Number (0x56190527 — this is the U-Boot magic 'IH_MAGIC'), CRC Checksum of the header, Image Creation Timestamp (Unix timestamp), and Size of Image in bytes. These are raw hex values in big-endian format.



```
U-Boot# md 0x82000000 10
82000000: 56190527 ba031e65 ca8df751 20c64100  '..Ue...Q....A.'
82000010: 00800080 00800080 9c919f36 00020205  '.....6.....'
82000020: 73676e41 6d6f7274 382e332f 2f33312e  'Angstrom/3.8.13/'
82000030: 67616562 6f62656c 0000656e 00000000  'beaglebone.....'
U-Boot#
```

 Fig 2.4 — Extended 'md 0x82000000 10' output showing 16 words (64 bytes = full header). The highlighted text on the right side shows 'Angstrom/3.8.13/beaglebone' this is the ih_name field decoded as ASCII. Reading the rows: Row 1 (magic, CRC, timestamp, size), Row 2 (load address 0x80008000, entry point 0x80008000, data CRC, OS/arch/type/comp bytes), Rows 3-4 (the 32-byte image name string).



```
## Checking Image at 82000000 ...
Legacy image found
Image Name:   Angstrom/3.8.13/beaglebone
Image Type:   ARM Linux Kernel Image (uncompressed)
Data Size:    4310560 Bytes = 4.1 MiB
Load Address: 80008000
Entry Point:  80008000
Verifying Checksum ... OK
U-Boot#
```

 Fig 2.5 — Using 'imi 0x82000000' command — the Image Info command. This is the proper way to decode the U-Boot header. It prints all fields in human-readable format: Image Name, Image Type (ARM Linux Kernel), Data Size (4.1 MiB), Load Address (0x80008000), Entry Point (0x80008000), and Checksum verification result (OK). This is much easier to read than raw hex from 'md'.

```
# The two commands to read uImage header:
md 0x82000000 4      # Raw hex dump of first 16 bytes (manual decoding)
imi 0x82000000       # Formatted image info (recommended — much clearer)
```


U-Boot Boot Log — What Success Looks Like

```
# After loading uImage to DDR RAM:
imi 0x82000000

# Example output:
# Image Name:      Linux-4.4.62-ti-r99
# Created:         2017-xx-xx
# Image Type:      ARM Linux Kernel Image (uncompressed)
# Data Size:       11334208 Bytes = 10.8 MiB
# Load Address:   80008000
# Entry Point:     80008000
# Verifying Checksum ... OK
```

 Fig 2.6 — Successful U-Boot kernel boot log in minicom. Reading line by line: '## Booting kernel from Legacy Image at 82000000' — U-Boot found the kernel at the DDR address; Image Name, Type, Size, Load Address, Entry Point are printed from the ulmage header; 'Verifying Checksum ... OK' — CRC check passed; '## Flattened Device Tree blob at 88000000' — DTB found at its DDR address; 'Loading Kernel Image ... OK'; 'Using Device Tree in place at 88000000'; 'Starting kernel ...' — this is U-Boot's

Important

'Starting kernel ...' is the last message printed by U-Boot.

Everything after that comes from the Linux kernel itself.

If the screen goes blank after 'Starting kernel ...' it means the kernel crashed or the console (ttyO0) is not configured correctly in bootargs.

SECTION 3 — uEnv.txt File

3. uEnv.txt — Automating U-Boot

Instead of typing boot commands manually at the U-Boot prompt every time, you can create a file called `uEnv.txt` and place it on the SD card's BOOT partition. U-Boot reads this file automatically and executes the commands inside it. This is how BBB automates the Linux boot process.

Note

`uEnv.txt` must be placed in the BOOT partition (FAT formatted, partition 1) of the SD card, NOT in the ROOTFS partition. U-Boot can only read FAT partitions; it cannot read ext3/ext4.

3.1 How uEnv.txt Works

When U-Boot starts its boot process (the 'boot' command), it automatically looks for a file named `uEnv.txt` on the boot device. If found, it reads the variables defined in the file and adds them to U-Boot's environment, then executes the 'uenvcmd' variable if it exists.

3.2 Complete uEnv.txt File Example and Explanation

```
console=ttyO0,115200n8
ipaddr=192.168.7.2
serverip=192.168.7.1
loadaddr=0x82000000
fdtaddr=0x88000000
loadfromsd=load mmc 0:2 ${loadaddr} /boot/uImage;load mmc 0:2 ${fdtaddr}
/boot/am335x-boneblack.dtb
linuxbootargs=setenv bootargs console=${console} root=/dev/mmcblk0p2 rw
uenvcmd=setenv autoload no; run loadfromsd; run linuxbootargs; bootm ${loadaddr} -
${fdtaddr}
```

Line-by-Line Explanation:

Variable	Explanation
console=ttyO0,115200n8	Sets the Linux kernel console to UART0 at 115200 baud, 8N1 (no parity, 1 stop bit). 'n8' means 'no parity, 8 data bits'.
ipaddr=192.168.7.2	Sets the IP address of the BBB board (used for TFTP/NFS networking).
serverip=192.168.7.1	Sets the IP address of the host PC (TFTP/NFS server).
loadaddr=0x82000000	A variable storing the DDR address where the kernel image will be loaded.

Variable	Explanation
fdtaddr=0x88000000	A variable storing the DDR address where the Device Tree Blob (DTB) will be loaded. 'fdt' = Flattened Device Tree.
loadfromsd=load mmc 0:2...	Loads ulmage from SD card (mmc 0, partition 2) to loadaddr, then loads DTB to fdtaddr. ';' chains two commands.
linuxbootargs=setenv bootargs...	Builds the kernel command line tells the kernel to use ttyO0 for console, mount mmcblk0p2 (SD partition 2) as root, read-write.
uenvcmd=...	The main auto-execute variable. Disables auto DHCP, runs loadfromsd, sets bootargs, then boots the kernel.

Note

mmc 0:2 breakdown:

- mmc 0 = First MMC device (SD card in the card slot)
- mmc 1 = Second MMC device (eMMC soldered to board)
- 0:2 = Device 0, Partition 2 (the ext3 ROOTFS partition)
- 0:1 = Device 0, Partition 1 (the FAT BOOT partition)
- mmcblk0p2 = Linux device name for SD card partition 2 (same as 0:2)

3.3 Creating uEnv.txt via U-Boot Shell (Serial Transfer)

If you need to test uEnv.txt over serial without an SD card, you can create the file on your PC, transfer it via xmodem/ymodem protocol, and import it into U-Boot memory:

```
# Create uEnv.txt on your PC with content like:
mycpip=setenv serverip 192.168.1.2

# In minicom, transfer the file using ymodem protocol:
# Press: Ctrl+A, then S → select 'ymodem' → navigate to file → Enter

# After transfer, note the address where it was loaded
# Then import its variables into U-Boot environment:
env import -t 0x82000000 36
# '36' = number of bytes in the file (check the transfer output)

# Verify the variable was imported:
printenv mycpip
# Output: mycpip=setenv serverip 192.168.1.2

# Execute the variable:
run mycpip
printenv serverip
# Output: serverip=192.168.1.2
```

SECTION 4 — SD Card Setup & Booting from SD

4. SD Card Setup for Booting BBB

To boot BBB from an SD card (instead of the default eMMC), you need to prepare the SD card with two specific partitions and copy the correct boot files to each partition.

4.1 Required SD Card Partition Layout

Partition	Details
Partition 1: BOOT	Format: FAT16 or FAT32 Label: BOOT Size: 1 GB Flag: bootable Contents: MLO, u-boot.img, uEnv.txt
Partition 2: ROOTFS	Format: ext3 Label: ROOTFS Size: Remaining space Contents: Extracted root filesystem (all Linux files)



Fig 4.1 — µSD Card Partitions diagram. The SD card must be divided into exactly two partitions: Partition 1 (BOOT) — formatted as FAT filesystem, holds boot images (MLO, u-boot.img, uEnv.txt). This is the partition U-Boot reads because U-Boot only understands FAT. Partition 2 (ROOTFS) — formatted as EXT3 or EXT4, holds the complete Linux root filesystem (all system files, libraries, init scripts, user applications). The arrow at the bottom points to Partition 1 with the label 'This partition holds boot images'.

Note

Why two partitions?

U-Boot can only read FAT filesystems (BOOT partition). The Linux kernel needs an ext3/ext4 filesystem for the root filesystem (ROOTFS partition). They must be separate because each has different filesystem format requirements.

4.2 Preparing the SD Card Using GParted (Ubuntu)

27. Connect SD card reader to PC.
28. Open a terminal and run: `dmesg` to identify the SD card device name (e.g., `/dev/sdb`).
29. Install GParted if not already: `sudo apt-get install gparted`
30. Open GParted and select the SD card from the device drop-down.
31. Delete all existing partitions: right-click each → Delete.
32. Create BOOT partition: right-click unallocated → New → Filesystem: fat16 → Label: BOOT → Size: 1024 MB → Add.
33. Create ROOTFS partition: right-click remaining unallocated → New → Filesystem: ext3 → Label: ROOTFS → Size: all remaining → Add.
34. Click Apply (green checkmark) to write changes.
35. Right-click BOOT partition → Manage Flags → tick 'boot' → Close.

```
# Alternative: Use command line with fdisk (for advanced users)
sudo fdisk /dev/sdb
# Then follow n (new), p (primary), partition numbers, sizes
# Then: t (type), change partition 1 to FAT16 (type 6)
# Then: a (bootable flag on partition 1)
# Then: w (write and exit)

# Format the partitions:
sudo mkfs.vfat -F 16 -n BOOT /dev/sdb1
sudo mkfs.ext3 -L ROOTFS /dev/sdb2
```

GParted Result — Correct Partition Configuration

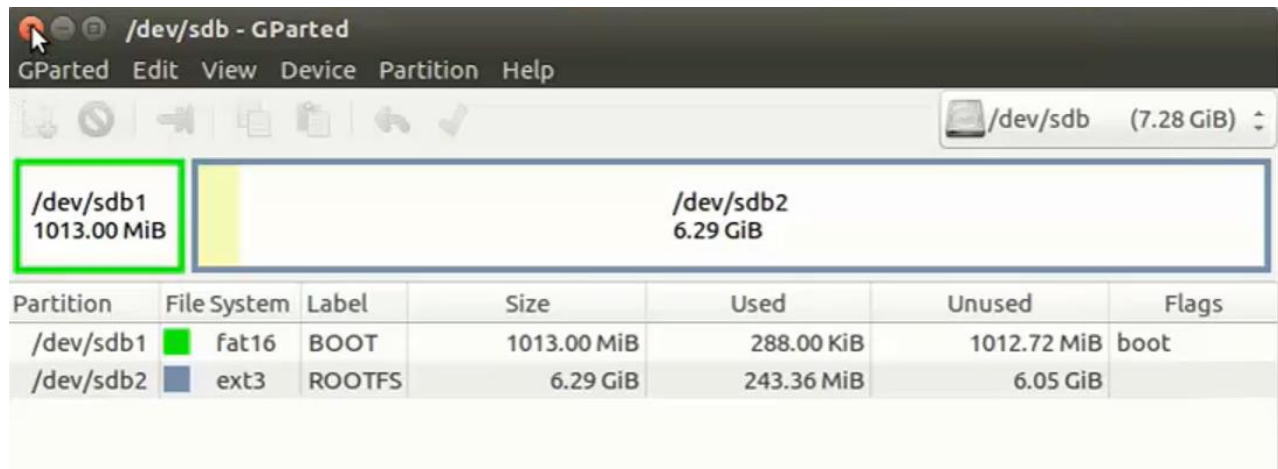


 Fig 4.2 — GParted showing the correctly partitioned SD card (`/dev/sdb`, 7.28 GiB total). The graphical bar at the top shows the two partitions visually. The table confirms: `/dev/sdb1` = fat16 filesystem, labelled BOOT, 1013 MiB, with the 'boot' flag set (critical — RBL only boots from a partition with this flag). `/dev/sdb2` = ext3 filesystem, labelled ROOTFS, 6.29 GiB remaining space. The 'boot' flag on `sdb1` is what tells the RBL this is the bootable partition.

4.3 Copying Files to SD Card

BOOT Partition Files (copy to /dev/sdb1 mount point):

File	Source / Notes
MLO	Rename 'MLO-beaglebone-2013.04' to 'MLO'. This exact name is required.
u-boot.img	Rename 'u-boot-beaglebone-2013.04-r0.img' to 'u-boot.img'. Exact name required.
uEnv.txt	Your custom boot configuration file (optional but recommended).

ROOTFS Partition Files (copy to /dev/sdb2 mount point):

```
# Mount ROOTFS partition
sudo mount /dev/sdb2 /mnt/rootfs

# Extract root filesystem (requires root for correct permissions)
sudo tar -xf Angstrom-systemd-image-eglibc-ipk-v2012.12-beagleboard.rootfs.tar.xz -C /mnt/rootfs

# Unmount when done
sudo umount /mnt/rootfs
```

Important

Always use 'sudo' when copying to SD card partitions.

Regular user permissions will cause 'permission denied' errors and the files won't copy correctly.

4.4 Booting BBB from SD Card

By default, BBB boots from the internal eMMC storage. To force it to boot from the SD card, use the S2 (boot) button:

36. Insert the prepared SD card into the BBB SD card slot.
37. Connect the UART-USB debug cable to your PC (for minicom serial output).
38. Open minicom on your PC: `minicom -D /dev/ttyUSB0 -b 115200`
39. Power off the BBB by long pressing the power button until the power LED turns off.
40. Press and HOLD the S2 button (boot button, located near the SD card slot).
41. While holding S2, press the power button to power on the board.
42. Release S2 after a few seconds — the board will now boot from SD card instead of eMMC.

Note

S2 is the small button next to the SD card slot on the BBB board.

It forces RBL to change its boot sequence to check the SD card first.

Without pressing S2, the board always boots from eMMC by default.

SECTION 5 — Linux Kernel Build

5. Building the Linux Kernel from Source

The Linux kernel is the core of the operating system. When building for BBB, we cross-compile it on our x86_64 PC to generate an ARM binary. The output is a 'ulmage' — a compressed kernel with a U-Boot header.

5.1 Download the Kernel Source

```
# Clone the BBB-specific kernel (v5.10.168 with TI real-time patches):  
git clone https://github.com/beagleboard/linux.git  
cd linux  
git checkout v5.10.168-ti-rt-r76
```

5.2 Kernel Build Steps

Step 1: Clean previous build artifacts

```
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- distclean
```

Step 2: Apply board default configuration

```
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- bb.org_defconfig
```

For kernel v4.11 and later, use: omap2plus_defconfig

This generates the .config file with all the driver selections for BBB.

Step 3: Customise kernel configuration (optional)

```
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- menuconfig
```

In the menuconfig UI:

[*] = built into kernel (compiled in)

[M] = loadable module (compiled as .ko file, loaded at runtime)

[] = disabled (not compiled at all)

Removing unnecessary drivers reduces the final kernel size.

Step 4: Compile the kernel and Device Tree

```
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabi- ulmage dtbs LOADADDR=0x80008000 -j4
```

ulmage = compressed kernel with U-Boot header

dtbs = Device Tree Blobs for all supported boards

LOADADDR = the physical RAM address where the kernel will run

Output: arch/arm/boot/ulmage and arch/arm/boot/dts/am335x-boneblack.dtb

Step 5: Compile loadable kernel modules

```
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabi- -j4 modules
```

Compiles all drivers selected as [M] in menuconfig into .ko files.

These files are needed to load device drivers at runtime.

Step 6: Install modules to root filesystem

```
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabi- INSTALL_MOD_PATH=<path/to/RFS>  
modules_install
```

Copies all .ko files into <RFS>/lib/modules/<kernel-version>/

Replace <path/to/RFS> with your root filesystem path, e.g., /srv/nfs/bbb



Note

LOADADDR=0x80008000 — this is where the kernel will be placed in physical RAM.

The U-Boot 'load' command copies the ulmage binary to the DDR address 0x82000000, but the kernel itself decompresses and relocates to 0x80008000 before running.

These are two different addresses and both must be correct for the kernel to boot.

SECTION 6 — Linux Device Tree

6. Linux Device Tree

The Device Tree is a data structure that describes the hardware of a board to the Linux kernel. Before Device Trees existed, hardware descriptions were hardcoded into the kernel source code — which meant maintaining thousands of board-specific kernel forks. Device Trees solved this problem.

6.1 What Problem Does the Device Tree Solve?

On x86 PCs, the BIOS/UEFI firmware discovers hardware automatically (via PCI, USB enumeration). On ARM embedded boards, there is no such automatic discovery — every peripheral (UART, SPI, I2C, GPIO, ethernet) is at a fixed memory address and the kernel must be told exactly where each one is.

The Device Tree is essentially a hardware description file. It says: 'There is a UART at address 0x44E09000, connected to interrupt 72, running at 48 MHz clock.' The kernel reads this file and configures the appropriate driver.

6.2 Device Tree Files and Formats

Extension	Description
.dts	Device Tree Source — human-readable text file that describes the hardware. Written by developers.
.dtsi	Device Tree Source Include — a shared fragment included by multiple .dts files (e.g., common SoC peripherals).
.dtb	Device Tree Blob — binary compiled version of the .dts file. This is what U-Boot loads and passes to the kernel.
dtc	Device Tree Compiler — tool that compiles .dts → .dtb or decompiles .dtb → .dts

6.3 Device Tree File for BBB

The main Device Tree file for BeagleBone Black is:

```
arch/arm/boot/dts/am335x-boneblack.dts
```

It includes the SoC-level file:

```
arch/arm/boot/dts/am335x.dtsi  (AM335x SoC peripherals)
arch/arm/boot/dts/am33xx.dtsi  (AM33xx family peripherals)
```

6.4 Basic Device Tree Syntax

```
/ {
    /* Root node - represents the entire system */

    model = "TI AM335x BeagleBone Black";
    compatible = "ti,am335x-bone-black", "ti,am335x-bone", "ti,am33xx";

    cpus {
        cpu@0 {
            compatible = "arm,cortex-a8";
        };
    };

    memory@80000000 {
        device_type = "memory";
        reg = <0x80000000 0x20000000>; /* 512 MB DDR starting at 0x80000000 */
    };

    uart0: serial@44e09000 {
        compatible = "ti,am3352-uart";
        reg = <0x44e09000 0x2000>; /* register base address, size */
        interrupts = <72>;
        status = "okay"; /* enabled */
    };
};
```

Note

The 'compatible' property is how the kernel matches a device node to a driver. The kernel's driver database is searched for a driver that lists 'ti,am3352-uart' in its compatible table. When found, that driver is used to control the UART. status = 'okay' means the device is enabled. status = 'disabled' means it's off.

6.5 How U-Boot Passes the DTB to the Kernel

U-Boot loads the .dtb file into DDR RAM at fdtaddr (typically 0x88000000) and then passes this address to the kernel as parameter r2 when calling kernel_entry(0, machid, r2). The kernel reads the hardware description from this memory location during boot.

```
# U-Boot: Load DTB and pass it to kernel
load mmc 0:2 ${fdtaddr} /boot/am335x-boneblack.dtb
bootm ${loadaddr} - ${fdtaddr}
#          ^kernel    ^ DTB address
```

SECTION 7 — initramfs (Initial RAM Filesystem)

7. initramfs — Initial RAM Filesystem

initramfs (Initial RAM File System) is a temporary, compressed root filesystem that is loaded into RAM during early kernel boot. It gives the kernel a minimal working environment before the actual root filesystem (on SD card / eMMC / NFS) is mounted.

7.1 Is initramfs Required?

No — initramfs is optional. Most desktop Linux distributions use it (to load drivers needed for the real disk). In embedded Linux, it is optional and commonly used when:

- The real root filesystem requires a driver that is not compiled into the kernel.
- You want to boot entirely in RAM (no SD card / eMMC needed).
- You need a recovery environment.

With initramfs	Without initramfs
Kernel boots → loads initramfs → runs /init from RAM → mounts real rootfs	Kernel boots → directly mounts real rootfs (SD/eMMC) → runs /sbin/init
Useful: loads missing drivers before mounting real rootfs	Simpler: fewer components, faster if all drivers are built-in
Required if root-disk driver is a loadable module	Works fine if all needed drivers are compiled into the kernel

7.2 What Does initramfs Contain?

- Device drivers (as loadable modules if needed)
- Firmware files for hardware initialisation
- /init script — the first program run from initramfs
- Essential utilities (mount, pivot_root, switch_root)
- Boot scripts and configuration files

7.3 Building initramfs — Step by Step

Step 1: Create cpio archive from your root filesystem directory

```
cd <your_rootfs_directory>
find . | cpio -H newc -o > ../initramfs.cpio
```

'cpio' = Copy In/Out. '-H newc' = new (SVR4) portable format. '-o' = output mode.

This creates a single archive file containing all files in the current directory.

Step 2: Compress the cpio archive with gzip

```
gzip < ../initramfs.cpio > ../initramfs.gz
```

Compresses the archive significantly (typically 3–5x smaller).

Step 3: Install U-Boot tools

```
sudo apt-get install u-boot-tools
```

This installs the 'mkimage' utility needed for Step 4.

Step 4: Add U-Boot header with mkimage

```
mkimage -A arm -O linux -T ramdisk -C none -a 0x80800000 -n "Root Filesystem" -d ../initramfs.gz  
../initramfs
```

-A arm = target architecture

-O linux = operating system

-T ramdisk = image type is ramdisk (initramfs)

-C none = not compressed at this stage (already gzipped)

-a 0x80800000 = load address in DDR RAM

-n "Root ..." = image name/description

-d ../initramfs.gz = input file

Output: 'initramfs' binary ready to be loaded by U-Boot

Booting with initramfs from U-Boot:

```
# Load kernel image  
loadx 0x82000000 # or: load mmc 0:1 0x82000000 uImage  
  
# Load DTB  
loadx 0x88000000 # or: load mmc 0:1 0x88000000 am335x-boneblack.dtb  
  
# Load initramfs  
loadx 0x88080000 # or: load mmc 0:1 0x88080000 initramfs  
  
# Set boot args: root=/dev/ram0 tells kernel to use initramfs as root  
setenv bootargs console=ttyO0,115200 root=/dev/ram0 rw initrd=0x88080000  
  
# Boot: note all three addresses are given  
bootm 0x82000000 0x88080000 0x88000000  
# ^kernel ^initramfs ^dtb
```


SECTION 8 — Booting BBB via Serial Port

8. Booting BeagleBone Black over Serial Port

If no SD card, USB, or SPI device is available, the BBB's RBL will fall back to UART boot mode. This allows you to load MLO, U-Boot, kernel, and DTB directly from your PC to the BBB using the xmodem serial transfer protocol via minicom.

8.1 How the RBL Selects UART Boot

The RBL scans boot sources in this fixed order: SPI → MMC0 (SD card) → USB → UART. When all other sources fail or are absent, it starts printing 'CCC...' on the UART — this is the xmodem CRC handshake signal, indicating it is ready to receive data.

How to Force UART Boot:

43. Remove SD card from BBB.
44. Do NOT connect any USB storage to the P4 mini-USB port.
45. Connect only the UART debug cable (UART0 RX/TX/GND pins) to your PC.
46. Open minicom on PC (baud rate 115200, 8N1).
47. Hold the S2 (boot) button and press power button. You will see 'CCCC...' in minicom.

8.2 Transferring Files via xmodem

Once you see 'CCC...' in minicom, transfer files using the xmodem protocol:

```
# In minicom, to send a file using xmodem:
# Press: Ctrl+A, then release, then press S
# A menu appears. Select 'xmodem' using arrow keys, press Enter
# Navigate to your file, press Space to select it, press Enter to start transfer
```

Complete Serial Boot Sequence:

48. See 'CCC...' — RBL is waiting. Send SPL binary (sp.bin or MLO.byteswap) via xmodem. SPL is written to internal SRAM.
49. SPL loads and prints messages. It now waits for U-Boot. Send u-boot.img via xmodem. U-Boot is written to DDR RAM.
50. U-Boot starts. Press Space bar to stop autoboot and get the U-Boot prompt.
51. Transfer kernel image to DDR RAM:

```
loadx 0x82000000
# Then send uImage via xmodem from minicom
```

52. Transfer DTB:

```
loadx 0x88000000
# Then send am335x-boneblack.dtb via xmodem
```

53. Transfer initramfs (optional):

```
loadx 0x88080000  
# Then send initramfs via xmodem
```

54. Set boot arguments and boot:

```
setenv bootargs console=ttyO0,115200 root=/dev/ram0 rw initrd=0x88080000  
bootm 0x82000000 0x88080000 0x88000000
```

 Tip

Serial boot is mostly useful for recovery or first-time board bring-up.

For regular development, TFTP boot (Section 9) is much faster because you don't have to manually transfer each file — TFTP downloads them over Ethernet.

SECTION 9 — TFTP Network Boot

9. TFTP Network Boot (Faster Development Workflow)

TFTP (Trivial File Transfer Protocol) allows U-Boot on BBB to download the kernel, DTB, and initramfs from a TFTP server running on your host PC over an Ethernet cable. This is the fastest way to test new kernel or rootfs changes during development — no SD card re-flashing needed.

Hardware Connection

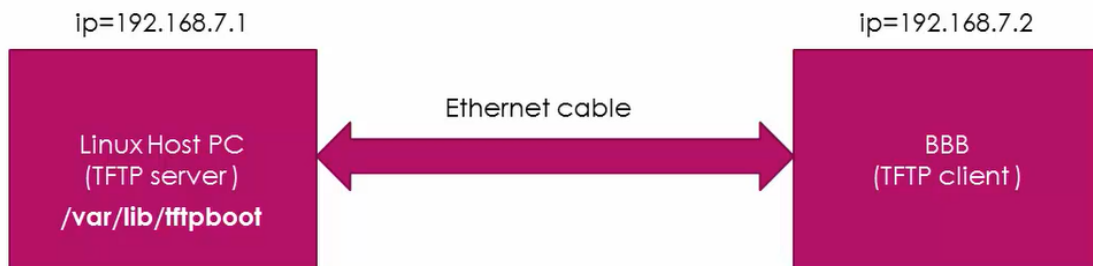


 Fig 9.1 — TFTP network connection diagram. Linux Host PC (left, IP=192.168.7.1) acts as the TFTP server — it runs `tftpd` and the kernel files are stored in `/var/lib/tftpboot` on this machine. BeagleBone Black (right, IP=192.168.7.2) acts as the TFTP client — U-Boot on BBB connects to the host PC and downloads the files. Both are connected with a direct Ethernet cable (crossover or standard, depending on your NICs). The bidirectional arrow shows TFTP data flows in both directions (request from BBB, data from PC).

9.1 Architecture Overview

The big picture: store MLO, u-boot.img, and uEnv.txt on eMMC to boot up to U-Boot, then transfer kernel and DTB from your PC using TFTP and mount the root filesystem from your PC using NFS.

Component	Location
MLO + u-boot.img	eMMC or SD card (FAT partition)
uEnv.txt	eMMC or SD card (FAT partition)
ulmage (kernel)	Host PC TFTP server directory: <code>/var/lib/tftpboot/</code>
am335x-boneblack.dtb	Host PC TFTP server directory: <code>/var/lib/tftpboot/</code>
initramfs	Host PC TFTP server directory: <code>/var/lib/tftpboot/</code>
Root Filesystem	Host PC NFS server directory: <code>/srv/nfs/bbb/</code>

9.2 Setting Up TFTP Server on Ubuntu

Step 1: Update and install TFTP daemon

```

sudo apt-get update
sudo apt-get install tftpd-hpa
  
```


Step 2: Configure the TFTP server

```
sudo vim /etc/default/tftpd-hpa  
Add/edit these lines:  
TFTP_USERNAME="tftp"  
TFTP_DIRECTORY="/var/lib/tftpboot"  
TFTP_ADDRESS=":69"  
TFTP_OPTIONS="--create --secure"
```

Step 3: Create and configure the TFTP directory

```
sudo mkdir -p /var/lib/tftpboot  
sudo chown tftp:tftp /var/lib/tftpboot  
sudo chmod -R 777 /var/lib/tftpboot  
Copy ulmage, am335x-boneblack.dtb, and initramfs to this directory.
```

Step 4: Start the TFTP service

```
sudo systemctl start tftpd-hpa  
sudo systemctl enable tftpd-hpa # start on boot
```

9.3 TFTP Boot Commands on BBB (at U-Boot prompt)

```
# Set network addresses  
setenv ipaddr 192.168.27.2 # BBB's IP address  
setenv serverip 192.168.27.1 # Host PC's IP address  
  
# Test connectivity  
ping 192.168.27.1  
  
# Download files from TFTP server to DDR RAM  
tftpboot 0x82000000 uImage  
tftpboot 0x88000000 am335x-boneblack.dtb  
tftpboot 0x88080000 initramfs  
  
# Set boot arguments  
setenv bootargs console=ttyO0,115200 root=/dev/ram0 rw initrd=0x88080000  
  
# Boot  
bootm 0x82000000 0x88080000 0x88000000
```

9.4 Setting Up NFS Root Filesystem

NFS (Network File System) allows BBB to mount its root filesystem from the host PC over the network. Combined with TFTP, this gives you a fully network-booted BBB with no local storage required — perfect for development.

Step 1: Install NFS server on host PC

```
sudo apt-get install nfs-kernel-server
```

Step 2: Configure NFS exports

```
sudo vim /etc/exports
```

Add this line:

```
/srv/nfs/bbb 192.168.27.2(rw,sync,no_subtree_check,no_root_squash)
```

This exports /srv/nfs/bbb directory to the BBB at 192.168.27.2



```
#  
/srv/nfs/bbb 192.168.7.2(rw,sync,no_root_squash,no_subtree_check)  
~ BBB ip address
```

 Fig 9.2 — /etc/exports file entry for NFS server. This single line exports the /srv/nfs/bbb directory to the BBB (IP 192.168.7.2) with options: *rw* (read-write access), *sync* (write operations complete before returning), *no_root_squash* (allows root on BBB to have root access to NFS files — needed for device files and permissions), *no_subtree_check* (performance improvement). After editing, run 'sudo exportfs -rv' to apply changes.

Step 3: Configure host PC network interface

```
sudo vim /etc/network/interfaces
```

Add entry for your Ethernet interface with static IP 192.168.27.1

Step 4: Apply NFS settings

```
sudo exportfs -a
```

```
sudo exportfs -rv
```

```
sudo service nfs-kernel-server restart
```

```
sudo service nfs-kernel-server status
```

NFS Boot Arguments (kernel bootargs):

```
setenv bootargs console=ttyO0,115200 root=/dev/nfs  
nfsroot=192.168.27.1:/srv/nfs/bbb,v3 rw ip=192.168.27.2
```

Tip

TFTP + NFS workflow for daily development:

1. Edit source code on your PC
2. Compile new kernel or application
3. Copy binary to `/var/lib/tftpboot/` or `/srv/nfs/bbb/`
4. Reboot BBB — it downloads fresh binaries automatically

No SD card re-flashing ever needed!

SECTION 10 — BusyBox

10. BusyBox — Minimal Root Filesystem

BusyBox is a single executable binary that contains minimised versions of over 300 Unix/Linux command-line utilities. It replaces the need for separate binaries like `ls`, `cp`, `mv`, `grep`, `sed`, `awk`, etc. BusyBox is the standard way to create a minimal root file system for embedded Linux products.

10.1 Why BusyBox?

Standard Linux (PC)	BusyBox (Embedded)
<code>/bin/ls</code> (separate binary)	<code>busybox ls</code> (same binary handles all commands)
<code>/bin/cp</code> (separate binary)	<code>busybox cp</code>
<code>/bin/grep</code> (separate binary)	<code>busybox grep</code>
Total <code>/bin</code> : ~162 commands as separate files	BusyBox <code>/bin</code> : 89 commands — 1 file, symlinked
<code>/bin</code> size: several MB	<code>busybox</code> binary: ~1.4 MB total for all commands
Cannot customise easily	Remove any command you don't need at compile time

Note

BusyBox uses symbolic links (symlinks) as a clever trick. The single 'busybox' binary checks `argv[0]` (the name it was called with) to decide which command to run. So 'ls' is just a symlink pointing to busybox. When you run 'ls', busybox sees its name is 'ls' and runs ls functionality. This saves enormous amounts of storage.

10.2 Compiling BusyBox

Download from: <https://busybox.net/downloads/>

Step 1: Apply default configuration

```
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- defconfig
```

Step 2: Customize (optional) — enable static build

```
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- menuconfig
```

Navigate to: BusyBox Settings → Build BusyBox as a static binary (no shared libs)

Select this option and save.

Why static? Embedded boards may not have the standard C library (glibc) installed.

A statically linked BusyBox has everything built in and runs anywhere.

Step 3: Compile and install to your rootfs directory

```
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- CONFIG_PREFIX=<install_path> install
```

Replace <install_path> with your workspace directory, e.g., /home/user/BBB_RFS

After install, your rootfs will contain:

bin/ → 89 commands (all symlinks to busybox)

sbin/ → 70 system commands (all symlinks to busybox)

usr/ → additional utilities

linuxrc → symlink to busybox (used with initramfs)

Verify: `du -sh <install_path>` → approximately 1.4 MB total

Step 4: Install kernel modules into rootfs

```
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- INSTALL_MOD_PATH=<install_path>  
modules_install
```

This creates <install_path>/lib/modules/<kernel-version>/kernel/drivers/

Also manually create: `mkdir <install_path>/dev` (empty device directory)

10.3 BusyBox Init System

BusyBox provides its own init program which becomes PID 1 (the first user space process). It reads `/etc/inittab` and runs scripts from `/etc/init.d/` in alphabetical order. Understanding this is crucial for embedded Linux system startup customization.

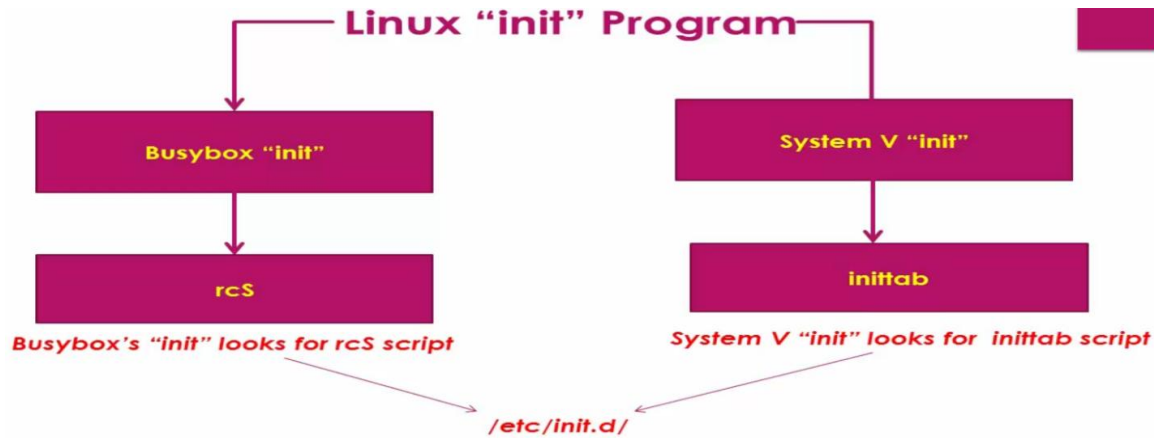


 Fig 10.1 — Linux 'init' Program — two implementations. LEFT branch: BusyBox 'init' — when BusyBox is PID 1, it reads `/etc/inittab` and then runs the `rcS` script from `/etc/init.d/`. `rcS` is the master startup script. RIGHT branch: System V 'init' — the traditional Linux init used on desktop systems, which reads `/etc/inittab` for service configuration. Both implementations ultimately look for scripts in `/etc/init.d/` (shown at bottom where both arrows converge). For embedded BBB with BusyBox, we use the LEFT branch: BusyBox init → `rcS` → individual `S##` scripts.

Script Name	Purpose
<code>rcS</code>	Master startup script — mounts <code>/proc</code> , runs all <code>S*</code> scripts in <code>/etc/init.d/</code>
<code>rcK</code>	Master shutdown script — runs all <code>K*</code> scripts to stop services
<code>S01Logging</code>	First to run — starts syslog (system log) and klog (kernel log)
<code>S02module</code>	Loads kernel modules (e.g., <code>g_ether</code> for USB-over-Ethernet)
<code>S04network</code>	Brings up network interfaces (runs <code>'ifup -a'</code>)
<code>S05sshd (or S50sshd)</code>	Starts the SSH daemon for remote access

Note

Script naming convention: S + two-digit number + name

S = Start script (run by `rcS` during boot)

K = Kill/stop script (run by `rcK` during shutdown)

Number = execution order (01 runs before 50)

Example: `S01Logging` runs before `S04network` which runs before `S50sshd`

Required Directories in rootfs:

```
mkdir -p etc/init.d      # init scripts
mkdir -p proc            # virtual filesystem for process info
mkdir -p sys            # virtual filesystem for hardware info
mkdir -p dev            # device files
mkdir -p etc/network    # network configuration
```

Loading Kernel Modules at Runtime:

```
# Manually load a kernel module:
modprobe g_ether
# modprobe is intelligent — it checks dependencies first

# List loaded modules:
lsmod

# After loading g_ether, USB0 network interface appears:
ifconfig usb0 192.168.6.2 up
ifconfig      # usb0 now visible
```

SECTION 11 — Buildroot

11. Buildroot — Automated Embedded Linux Build System

Buildroot is a tool that automates everything we have done manually in the previous sections. Instead of individually downloading and compiling U-Boot, the Linux kernel, BusyBox, and third-party packages, Buildroot does it all with a single 'make' command. It generates a complete, ready-to-use embedded Linux image.

11.1 What Buildroot Can Build

- Cross-compilation toolchain (or use an external one like Linaro)
- Bootloader (U-Boot, MLO/SPL)
- Linux kernel (with Device Tree Blobs)
- Root filesystem (using BusyBox, plus optional packages)
- Additional packages: OpenSSH, Python, web servers, databases, and hundreds more



Tip

Think of Buildroot as a 'recipe book' for your embedded Linux system.

You tell it what hardware you have (BBB), what features you want (SSH, specific kernel version), and it downloads, compiles, and packages everything automatically.

11.2 Getting Started with Buildroot

Download from: <https://buildroot.org/download.html>

Directory	Contents
configs/	Pre-made configurations for popular boards (beaglebone_defconfig is here)
board/	Board-specific files, post-build scripts, and READMEs
board/beaglebone/	BBB-specific overlay files and build scripts
board/beaglebone/readme.txt	Step-by-step instructions for building for BBB
package/	Package recipes for hundreds of software packages
output/images/	All compiled output files appear here after build

11.3 Configuring and Building for BBB

Step 1: Load BBB default configuration

```
cd buildroot-2017.05
make beaglebone_defconfig
```

Step 2: Open configuration menu

```
make menuconfig
```

Key menuconfig Settings for BBB:

Menu Path	Setting
Toolchain → Toolchain Type	Select 'External toolchain' (to use your Linaro installation)
Toolchain → Toolchain	Linaro (auto-detected if installed)
Toolchain → Toolchain origin	Pre-installed toolchain
Toolchain → Toolchain path	Path to your toolchain bin/ directory
System configuration → System hostname	Your custom hostname (e.g., my-bbb)
System configuration → System banner	Welcome message (e.g., Welcome to Buildroot-BBB)
System configuration → Init system	BusyBox
System configuration → Root password	Set a root password
System configuration → getty (login prompt)	ttyO0 at 115200 baud
Linux kernel → Defconfig name	bb.org (or omap2plus for kernel 4.11+)
Linux kernel → Kernel binary format	ulmage
Linux kernel → Load address	0x80008000
Linux kernel → Device tree source	am335x-boneblack
Target packages → Networking → openssh	Enable for SSH access
Bootloader → U-Boot	Disable (we already have our own)

Step 3: Build everything

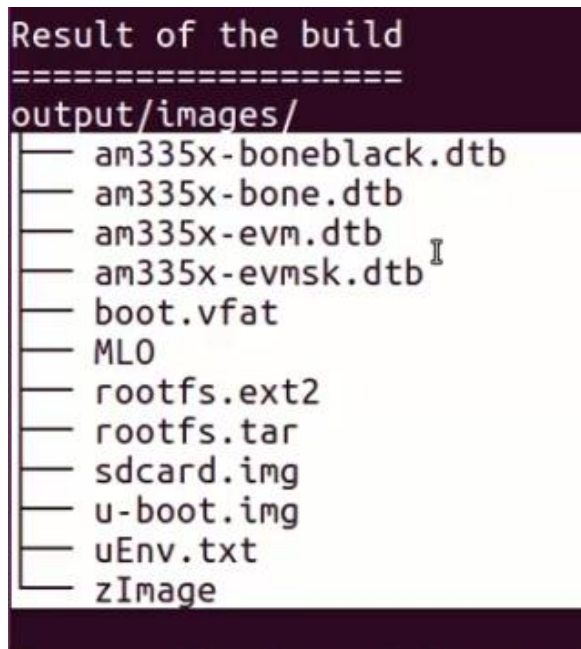
make -j4

This can take 30–90 minutes on first run (downloads and compiles everything).

Output files in buildroot-2017.05/output/images/:

MLO, u-boot.img, ulmage, am335x-boneblack.dtb, rootfs.tar

Buildroot Output Files



```
Result of the build
=====
output/images/
— am335x-boneblack.dtb
— am335x-bone.dtb
— am335x-evm.dtb
— am335x-evmsk.dtb
— boot.vfat
— MLO
— rootfs.ext2
— rootfs.tar
— sdcard.img
— u-boot.img
— uEnv.txt
— zImage
```

 Fig 11.1 — Result of the Buildroot build showing output/images/ directory contents. After 'make -j4' completes, these files are generated: am335x-boneblack.dtb (Device Tree for BBB), am335x-bone.dtb, am335x-evm.dtb, am335x-evmsk.dtb (DTBs for other AM335x boards), boot.vfat (FAT boot partition image), MLO (Secondary Program Loader), rootfs.ext2 (root filesystem in ext2 format), rootfs.tar (root filesystem as tarball — use this to extract to NFS), sdcard.img (complete SD card image ready to dd), u-boot.img (U-Boot bootloader), uEnv.txt (auto-generated boot script), zImage (compressed kernel). The sdcard.img is the most convenient — you can dd it directly to an SD card.

11.4 Deploying Buildroot Output

```
# Remove old rootfs from NFS directory
sudo rm -rf /srv/nfs/bbb/*

# Extract new rootfs
sudo tar -xf output/images/rootfs.tar -C /srv/nfs/bbb

# Copy kernel and DTB to TFTP directory
cp output/images/uImage /var/lib/tftpboot/
cp output/images/am335x-boneblack.dtb /var/lib/tftpboot/

# Reboot BBB and it will use the new images
```

11.5 Enabling SSH in Buildroot

After enabling OpenSSH in menuconfig and rebuilding, you may need to configure the sshd config on the target:

```
# On host PC, edit the rootfs (before extracting) or after SSH fails:
vi /srv/nfs/bbb/etc/ssh/sshd_config

# Uncomment or add:
PermitRootLogin yes

# Restart NFS service:
sudo service nfs-kernel-server restart

# From host PC, SSH into BBB:
ssh -l root 192.168.7.2
# Enter root password when prompted

# From BBB, SSH into host PC:
ssh kiran@192.168.7.1
```

SECTION 12 — Linux Source Tree & Boot Flow

12. Linux Source Tree & Detailed Boot Flow

Understanding the Linux source tree structure and the detailed boot flow is essential for any embedded Linux developer. This section maps the boot sequence to exact source files.

12.1 Linux Kernel Source Tree — Key Directories

Directory/File	Contents & Purpose
arch/	Architecture-specific code (arm, x86, mips, riscv, arm64, etc.)
arch/arm/boot/	ARM boot files: head.S, compressed/, dts/
arch/arm/boot/compressed/head.S	FIRST file executed after U-Boot hands over control. Decompresses kernel.
arch/arm/boot/compressed/misc.c	Kernel decompression logic — unzips the compressed kernel image
arch/arm/kernel/head.S	SECOND head.S — CPU init, MMU setup, calls start_kernel()
arch/arm/kernel/head-common.S	Calls the C main function after assembly setup completes
arch/arm/lib/bootm.c	U-Boot file: implements boot_jump_linux() to call kernel entry
arch/arm/mm/	ARM memory management (page tables, cache, TLB)
arch/arm/boot/dts/	All Device Tree source files for ARM boards
init/main.c	Architecture-independent kernel main — start_kernel() is here
kernel/	Core kernel: scheduler, timers, signals, modules
mm/	Memory management subsystem
drivers/	All hardware device drivers (tens of thousands of files)
fs/	Filesystem implementations (ext4, fat, nfs, proc, sys, etc.)
net/	Network protocols (TCP/IP, UDP, NFS, etc.)
include/	Kernel header files
Documentation/	Extensive kernel documentation
Makefile	Top-level build system
Kconfig	Configuration system (what menuconfig reads)
.config	Generated after menuconfig — contains all kernel settings

12.2 Detailed Boot Flow: U-Boot to Linux User Space

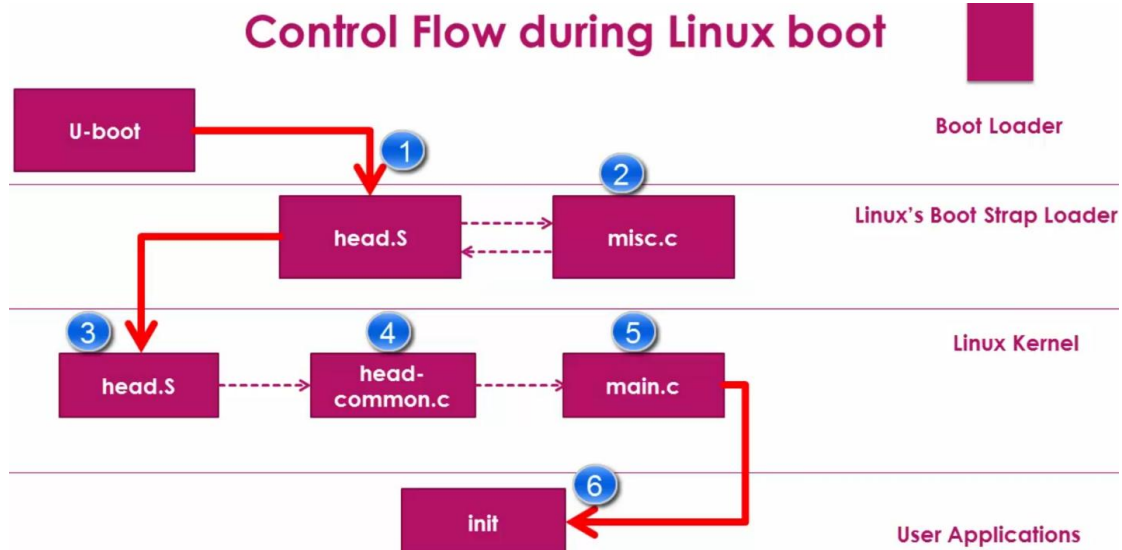


Fig 12.1 — Control Flow during Linux Boot (source file level diagram). U-Boot (top-left) calls into Linux's Boot Strap Loader layer (step 1): `arch/arm/boot/compressed/head.S` runs first, which calls `misc.c` (step 2) to decompress the kernel, then `head.S` regains control. Step 3: the second `head.S` at `arch/arm/kernel/head.S` runs (CPU/MMU initialisation). Step 4: `head-common.S` bridges to C code. Step 5: `init/main.c` `start_kernel()` function is called. Step 6: `start_kernel()` eventually launches the `init` process (PID 1) in User Applications layer.

#	Source File	What Happens
1	<code>arch/arm/lib/bootm.c</code> (U-Boot)	<code>boot_jump_linux()</code> creates <code>kernel_entry</code> function pointer. Calls <code>kernel_entry(0, machid, r2)</code> . <code>r2</code> = DTB address in DDR RAM.
2	<code>arch/arm/boot/compressed/head.S</code>	First kernel assembly code runs. Sets up basic environment, prepares for decompression.
3	<code>arch/arm/boot/compressed/misc.c</code>	Decompresses the kernel image from its compressed format to plain binary in RAM.
4	<code>arch/arm/kernel/head.S</code>	CPU-specific initialisation, validates processor architecture, sets up page tables, enables MMU.
5	<code>arch/arm/kernel/head-common.S</code>	Final setup, then calls the C function <code>start_kernel()</code> .
6	<code>init/main.c: start_kernel()</code>	Arch-independent kernel init: memory, interrupts, scheduler, filesystems, devices all initialised.
7	<code>init/main.c: rest_init()</code>	Creates two kernel threads: <code>kernel_init</code> (PID 1 candidate) and <code>kthreadd</code> (PID 2, kernel thread manager).
8	<code>kernel_init</code> thread	Frees <code>__init</code> memory. Tries to <code>exec /sbin/init</code> → <code>/etc/init</code> → <code>/bin/init</code> → <code>/bin/sh</code> in order.
9	<code>/sbin/init</code> or <code>busybox init</code>	PID 1 starts. Reads <code>/etc/inittab</code> , runs <code>/etc/init.d/rcS</code> , starts all services.

12.3 kernel_init — Finding PID 1

The kernel_init thread tries to find and run the first userspace program in this exact order:

```
/* From init/main.c */
if (execute_command) {
    run_init_process(execute_command); /* init= kernel argument */
}
run_init_process("/sbin/init");
run_init_process("/etc/init");
run_init_process("/bin/init");
run_init_process("/bin/sh");
/* If ALL fail: */
panic("No init found. Try passing init= option to kernel");
```

Note

You can specify a custom init program by passing 'init=<path>' in the kernel bootargs.

Example: setenv bootargs ... init=/bin/myapp

This is useful in embedded systems where the default /sbin/init is not needed.